

NAME(S):

Note: For the first 3 parts of this assignment, please draw the memory diagrams by hand using a piece of paper, a tablet, the white boards in class, etc.

Part 1. Tracing (re-)assignment with Memory Diagrams

Consider the following snippet of Python code.

```
1. x = 5
2. y = 10
3. z = x
4. x = y + 3
5. y = z
```

1. Draw a memory diagram that shows the <variable name>-<object/value> associations after line 3 has been executed.

2. Draw a memory diagram that shows the <variable name>-<object/value> associations after the whole snippet has been executed.

Part 2. Tracing loops with Memory Diagrams

Consider the following snippet of Python code.

```
1. x = int(input("Please enter a positive integer: "))
2. counter = x
3. for step in range(0,10):
4.     print(counter)
5.     counter += x
```

Note: `counter += 1` does the same thing as `counter = counter + 1`

1. Draw a memory diagram after line 2 has been executed. What the user enters is up to you, but keep it in mind for the following questions.
2. Draw a memory diagram after line 5 has been executed the first time.
3. Draw a memory diagram after line 5 has been executed the second time.
4. Draw a memory diagram at the very end of the program.

Part 3. Debugging with Memory Diagrams

The following program is supposed to add up numbers entered by the user. Unfortunately, it doesn't work correctly.

```
1. steps = int(input("How many numbers do you want to add up? "))
2. total = 0
3. number = float(input("Please enter the next number to add: "))
4. for step in range(0, steps):
5.     total = total + number
6.
7. print("Total: ", total)
```

1. Draw a memory diagram after line 2 has been executed. Assume that the user wants to add 4 numbers together.

2. Draw a memory diagram after line 5 has been executed the first time.

3. Draw a memory diagram after line 5 has been executed the second time.

4. Describe what the problem is with the program and how to fix it.

Part 4. Using Python Tutor

Python Tutor is a web service that visualizes memory diagrams for Python code. It may prove helpful for drawing memory diagrams and debugging your code.

1. Please open the following web page: <https://pythontutor.com/visualize.html>
2. Python Tutor supports different visualization styles, but the one closest to what we use in class, select “render all objects on the heap” using the second drop down below the code window.
3. Copy the code from page 1 into the code area.
4. Click Visualize Execution and use the Next button to step through each line of code. The memory diagram will appear on the right.
5. (Optional) Repeat steps 3 and 4 for the code on pages 2 and 3.

A link to Python Tutor can be found in the Resources page on the course website.